30-Day Node.js, Express.js & MongoDB Roadmap

A step-by-step learning and teaching plan

Goal	Build and explain a structured REST API with authentication, database integration, testing, and deployment.
Daily study time	Recommended: 2–3 hours per day.
Learning method	Understand concepts → code examples → practice → debugging → teach-back.
Final project	Course Management REST API with users, courses, and enrollments.

Month at a glance

Week	Focus	Days
1	JavaScript and Node.js fundamentals	1–7
2	Express.js and REST APIs	8–14
3	MongoDB, Mongoose, and CRUD integration	15–21
4	Authentication, security, testing, deployment, and project	22–30

Important: This is an intensive one-month plan. Aim to understand and build the core project; advanced topics will need continued practice after Day 30.

Week 1 — JavaScript and Node.js

Weekly goal: Understand JavaScript execution, asynchronous programming, modules, and HTTP servers.

Day 1: JavaScript revision for backend development

- Variables, data types, operators, conditions, and loops
- Functions, arrow functions, scope, and closures
- Arrays, objects, destructuring, spread/rest, and array methods
- JSON and JavaScript objects

Practice / deliverable: Create a student array and filter students by marks.

Day 2: How Node.js works

- Browser JavaScript vs Node.js; runtime and V8 engine
- Node.js APIs, event loop, and non-blocking I/O
- Synchronous vs asynchronous execution
- Callbacks, promises, and async/await revision

Practice / deliverable: Predict the execution order of synchronous code, promises, and timers.

Day 3: Node.js setup and npm

- Install Node.js LTS and verify versions
- Run JavaScript files using node
- npm init, package.json, scripts, dependencies, and devDependencies
- node_modules and package-lock.json

Practice / deliverable: Create a project with start and dev scripts.

Day 4: Modules and code organization

- CommonJS: require and module.exports
- ES modules: import and export
- Built-in, local, and third-party modules
- process.env, .env files, and configuration basics

Practice / deliverable: Split a calculator into reusable modules.

Day 5: Core Node.js modules

- File system: fs and fs/promises
- Paths using path
- Reading and writing files; handling file errors
- Working with JSON files

Practice / deliverable: Build a file-based student record manager.

Day 6: HTTP and the Node.js server

- Client-server architecture; HTTP requests and responses
- Headers, status codes, and HTTP methods
- Create a server with Node's http module
- Return JSON responses

Practice / deliverable: Build a basic server with two endpoints without Express.

Day 7: Revision and Node.js mini-project

- Revise modules, asynchronous code, and HTTP
- Debug common Node.js errors
- Organize files into reusable functions
- Explain the request-response lifecycle

Practice / deliverable: Deliverable: a Node.js HTTP server that returns student data.

Week 2 — Express.js and REST APIs

Weekly goal: Build structured API routes with middleware, validation, and error handling.

Day 8: Express.js introduction

- Why Express is useful; install Express
- Start a server with app.listen()
- Use app.get(), res.send(), and request/response objects
- Test endpoints with Postman or an equivalent API client

Practice / deliverable: Create /, /about, and /students endpoints.

Day 9: Routing and HTTP methods

- GET, POST, PUT, PATCH, and DELETE
- Route parameters with req.params
- Query strings with req.query
- Request bodies with req.body and express.json()

Practice / deliverable: Build CRUD routes using an in-memory array.

Day 10: Express middleware

- What middleware is and how request flow works
- req, res, and next
- Application-level and router-level middleware
- Built-in/third-party middleware, logging, and request timing

Practice / deliverable: Create a request logger and a simple API-key-checking middleware.

Day 11: REST API design

- Resources and resource-based URLs
- HTTP methods and status codes
- Consistent JSON response formats
- Pagination, filtering, sorting, naming, and versioning

Practice / deliverable: Design a REST API for courses and students.

Day 12: Validation and error handling

- Validate required fields and data types
- Handle invalid IDs and missing resources
- try/catch and asynchronous errors
- Centralized error-handling middleware
- Avoid exposing internal errors to clients

Practice / deliverable: Return consistent errors for invalid student input.

Day 13: Express project structure

- Routes, controllers, services, and middleware
- Separate app setup from server startup
- Use configuration files
- Keep business logic out of route handlers

Practice / deliverable: Refactor the student API into separate files.

Day 14: Express mini-project and assessment

- Complete in-memory CRUD API
- Test success and failure cases
- Add validation, middleware, and error handling
- Review HTTP status codes and REST conventions

Practice / deliverable: Deliverable: a structured student management API ready for MongoDB.

Week 3 — MongoDB and Mongoose

Weekly goal: Store persistent data, model relationships, and integrate CRUD endpoints.

Day 15: MongoDB fundamentals

- Relational databases vs NoSQL databases
- Databases, collections, and documents
- BSON, JSON, ObjectId, and document structure
- MongoDB Atlas or local installation

Practice / deliverable: Create a database and insert student documents.

Day 16: MongoDB CRUD and queries

- insertOne() and insertMany()
- find() and findOne()
- updateOne(), updateMany(), deleteOne(), and deleteMany()
- Filters and comparison operators

Practice / deliverable: Add, search, update, and delete students in MongoDB.

Day 17: Advanced MongoDB queries

- Projection and sorting
- Pagination with limit() and skip()
- $in, $or, $and, and $regex
- Update operators such as $set and $inc
- Basic aggregation pipeline concepts

Practice / deliverable: Filter students by marks, sort results, and paginate output.

Day 18: Mongoose fundamentals

- Native MongoDB driver vs Mongoose
- Connect with mongoose.connect()
- Schemas, models, and documents
- Data types, defaults, required fields, and validation

Practice / deliverable: Create a Student schema and model.

Day 19: Mongoose CRUD

- Model.create() and Model.find()
- findById() and findByIdAndUpdate()
- findByIdAndDelete()
- Validation errors and missing documents
- runValidators and update options

Practice / deliverable: Replace the in-memory student array with MongoDB persistence.

Day 20: Database relationships and modeling

- Embedded documents vs references
- One-to-one, one-to-many, and many-to-many concepts
- ObjectId references and populate()
- Data-modeling trade-offs, unique constraints, and basic indexes

Practice / deliverable: Model users, courses, and student course enrollments.

Day 21: Full backend database integration

- Connect Express to MongoDB
- Use Mongoose in controllers and services
- Add pagination, filtering, and sorting
- Handle database connection failures
- Test all CRUD endpoints

Practice / deliverable: Deliverable: a persistent course and student API.

Week 4 — Security and final project

Weekly goal: Add authentication, tests, deployment, documentation, and teaching practice.

Day 22: Authentication fundamentals

- Authentication vs authorization
- User registration and login
- Password hashing with bcrypt
- Never store plain-text passwords
- Login validation and safe error responses

Practice / deliverable: Implement registration with hashed passwords.

Day 23: JWT authentication

- JWT structure: header, payload, and signature
- Token signing, verification, and expiration
- Authentication middleware and protected routes
- Secure token storage and secret management

Practice / deliverable: Create login and protected /me endpoints; never put secrets in JWT payloads.

Day 24: Authorization and API security

- Role-based access: admin, instructor, and student
- Ownership checks and permission middleware
- Input validation and sanitization
- CORS, rate limiting, and security headers
- Protect sensitive fields and avoid mass assignment

Practice / deliverable: Restrict course creation to instructors or admins.

Day 25: Production-quality architecture

- Route, controller, service, and model responsibilities
- Configuration and environment validation
- Centralized error handling and structured logging
- Async operations and database connection lifecycle
- Graceful shutdown and clean code practices

Practice / deliverable: Refactor the project into a maintainable folder structure.

Day 26: API testing

- Unit tests vs integration tests
- Test runners and assertions
- Test success and failure responses
- Test validation, authentication, and authorization
- Use a separate test database

Practice / deliverable: Write automated tests using Jest or Vitest with Supertest.

Day 27: Performance and database reliability

- Indexes and query efficiency
- Avoid unnecessary database queries
- Pagination and response size
- Transactions and data consistency concepts
- Logging and troubleshooting slow endpoints

Practice / deliverable: Optimize a filtered course-list endpoint.

Day 28: Deployment and production configuration

- Prepare production scripts
- Configure environment variables and database access
- Deploy to a Node-compatible hosting provider
- Configure CORS and HTTPS
- Check logs, health endpoints, and deployment failures

Practice / deliverable: Deploy the API and verify its health endpoint.

Day 29: Final project completion

- Finish user, course, and enrollment APIs
- Add validation and role-based permissions
- Test end-to-end flows
- Write OpenAPI/Swagger docs or a Postman collection
- Fix bugs and prepare a project demonstration

Practice / deliverable: Deliverable: a documented and tested course-management backend.

Day 30: Revision, assessment, and teaching demonstration

- Revise Node.js, Express, and MongoDB concepts
- Explain request lifecycle and middleware flow
- Demonstrate authentication and database CRUD
- Solve debugging and interview questions
- Deliver a short classroom lesson and assess understanding

Practice / deliverable: Final outcome: working project, clear architecture explanation, and teaching-ready examples.

Final Project — Course Management REST API

Use one project throughout the month instead of building unrelated examples. Add features gradually as you learn each topic.

- User registration and login
- Role-based access for admins, instructors, and students
- Course create, list, update, and delete endpoints
- Student enrollment in courses
- Search, filtering, sorting, and pagination
- Input validation and consistent error responses
- MongoDB persistence using Mongoose
- Automated tests and API documentation
- Deployment to a Node.js-compatible host

Suggested project structure

```
course-management-api/
├── src/
│   ├── config/database.js
│   ├── models/User.js, Course.js, Enrollment.js
│   ├── controllers/
│   ├── services/
│   ├── routes/
│   ├── middleware/
│   ├── validators/
│   ├── app.js
│   └── server.js
├── tests/
├── .env
├── .gitignore
└── package.json
```

Suggested daily routine (2 hours 30 minutes)

Activity	Time	Purpose
Concept learning	40 min	Theory, diagrams, execution flow
Hands-on coding	60 min	Write and run examples
Practice and debugging	30 min	Solve exercises independently
Teaching preparation	20 min	Examples, questions, explanations

Daily completion checklist

For each day, check off these items in your own notes:

[ ] I can explain the concept in my own words.

[ ] I wrote and ran the example myself.

[ ] I completed the practice task.

[ ] I tested at least one edge case or common error.

[ ] I can explain the topic as if teaching a student.